



Práctica 1:

Modelo de McCulloch y Pitts 1ra Neurona Artificial

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: 6CM3

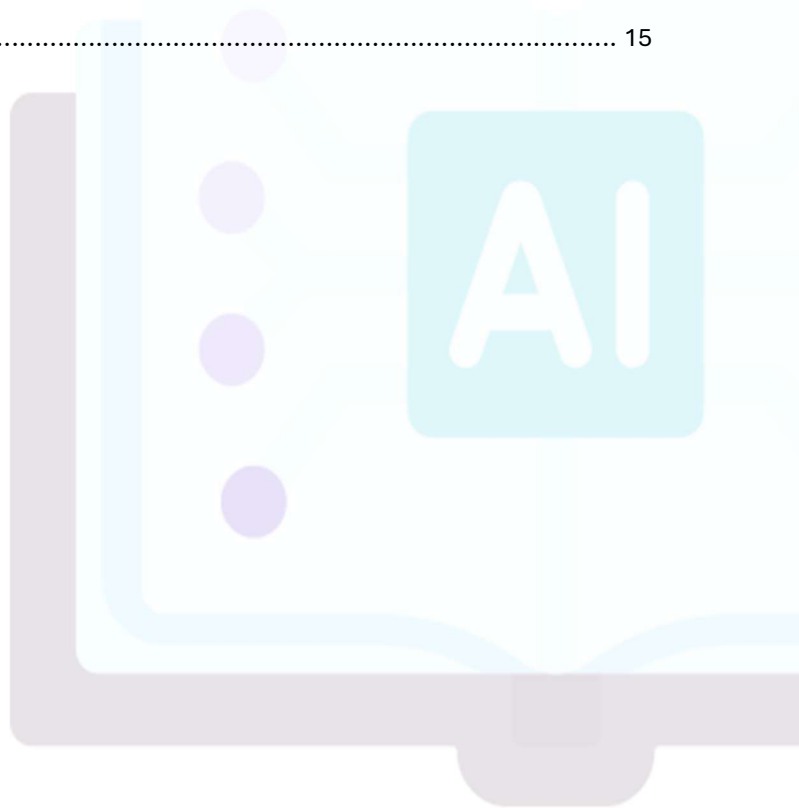
Nombres de los integrantes:

- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Viernes, 19 de septiembre, 2025

CONTENIDO

MARCO TEÓRICO	3
MAPA MENTAL.....	3
DESARROLLO.....	4
EJERCICIOS.....	4
NEURONAS DE MCCULLOCH Y PITTS	5
<i>Ejemplo de Código (comentado)</i>	5
<i>Neuronas Compuertas Lógicas</i>	6
<i>Neurona Compuerta XOR</i>	10
<i>Neurona medio sumador Half Adder</i>	12
CONCLUSIONES	14
AGUIRRE LANTO VÍCTOR MANUEL.....	14
GASCA FRAGOSO PEDRO	14
GUEVARA BADILLO ARELI ALEJANDRA	14
MONTIEL TORO ARAEL DE JESÚS.....	14
RAMÍREZ LOZANO GAEL MARTIN.....	15

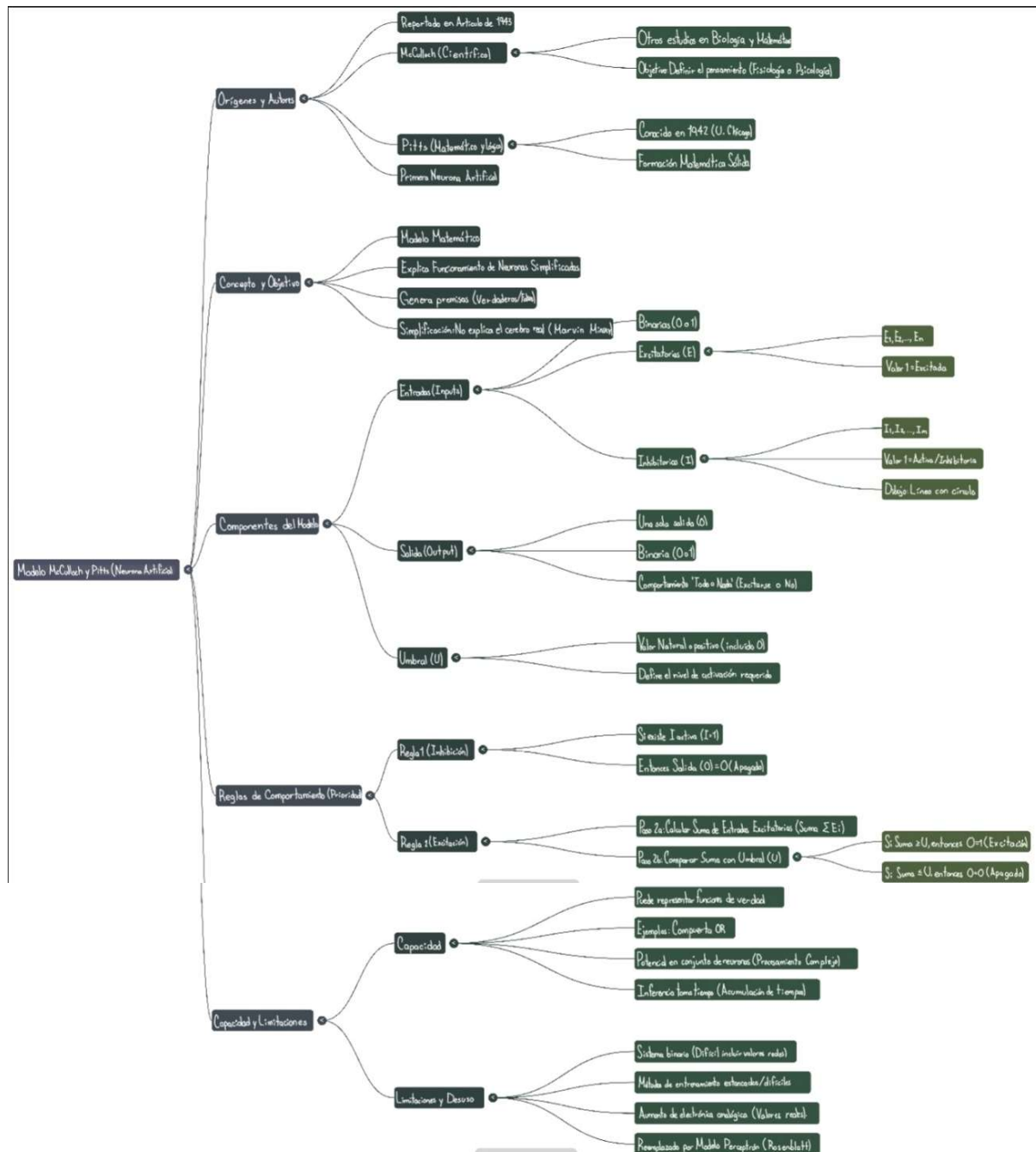


Marco Teórico

Las neuronas de McCulloch-Pitts, nombradas así por Warren McCulloch y Walter Pitts en 1943, son un modelo simplificado de neuronas biológicas. Estas neuronas artificiales fueron diseñadas para capturar las propiedades esenciales de las neuronas en el cerebro, permitiendo el estudio de sistemas nerviosos complejos a través de modelos matemáticos y computacionales.

Mapa Mental

A continuación, mostramos el Mapa Mental creado sobre el material proporcionado en Youtube:



Desarrollo

El modelo de McCulloch y Pitts se basa en la idea de que una neurona puede estar en uno de dos estados, activada o desactivada, lo que equivale a generar un impulso o no. La decisión de activarse se toma a través de la suma ponderada de sus entradas; si esta suma excede un cierto umbral, la neurona se activa (emite un valor de salida), de lo contrario permanece inactiva. Este concepto fue revolucionario porque permitió la aplicación de lógica binaria y álgebra booleana en el modelado del cerebro, proporcionando una de las bases para el desarrollo de las redes neuronales artificiales y, por extensión, del campo de la inteligencia artificial.

Ejercicios

A continuación, presentamos las soluciones de ambos ejercicios propuestos:

Exercise 3.1 Dada la neurona de MyP, $C = (E = [x_1, x_2], I = [i_1], u = 1)$, obtenga su diagrama gráfico y la tabla de salidas.

Ejercicio 3.1 Dada la neurona de MyP, $C = (E = \{x_1, x_2\}, I = \{i_1\}, u = 1)$, obtenga su diagrama gráfico y la tabla de salidas.

e_1	e_2	i_1	s
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Exercise 3.2 Diseñe una neurona de MyC de tal forma que cuando al menos dos de tres entradas se encuentren activas la salida se active. La salida debe estar apagada para cualquier otro caso.

Ejercicio 3.2 Diseñe una neurona de MyC de tal forma que cuando al menos dos de tres entradas se encuentren activas la salida se active. La salida debe estar apagada para cualquier otro caso.

e_1	e_2	e_3	s
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Neuronas de McCulloch y Pitts

Ejemplo de Código (comentado)

```
1 def neurona_mcculloch_pitts(E, I, u):
2     """
3     Simula una neurona de McCulloch y Pitts.
4
5     Parámetros:
6         E : lista de enteros (0 o 1) → entradas excitatorias
7         I : lista de enteros (0 o 1) → entradas inhibitorias
8         u : entero (umbral)
9
10    Retorna:
11        0 → la neurona permanece apagada
12        1 → la neurona se activa
13    """
14
15    # 1. Revisar si alguna entrada inhibitoria está activa (1).
16    # En el modelo de McCulloch-Pitts, cualquier inhibición anula la activación.
17    if 1 in I:
18        return 0 # Se apaga la neurona inmediatamente
19
20    # 2. Calcular la suma de las entradas excitatorias.
21    # Cada "1" cuenta como una señal de activación.
22    suma = sum(E)
23
24    # 3. Comparar la suma de excitatorias con el umbral.
25    # - Si la suma es mayor o igual al umbral → la neurona se activa.
26    # - Si no alcanza el umbral → la neurona permanece apagada.
27    if suma >= u:
28        return 1
29    else:
30        return 0
31
32
33    # Ejemplo de uso:
34    E = [1, 0, 1, 1] # entradas excitatorias
35    I = [0, 0] # entradas inhibitorias
36    u = 3 # umbral
37
38    salida = neurona_mcculloch_pitts(E, I, u)
39    print("Salida de neurona:", salida)
40
41    """
42    Explicación del ejemplo:
43    - La neurona recibe 3 excitaciones activas (1,0,1,1 → suma = 3).
44    - No hay inhibidores activos.
45    - Como la suma (3) es igual al umbral (3) → la neurona se activa (salida = 1).
46    """
47
48
```


Neuronas Puertas Lógicas

```
def neurona_mcculloch_pitts(E, I, u):
    # 1. Revisar entradas inhibitorias: si alguna está activa, la neurona no dispara
    if 1 in I:
        return 0

    # 2. Calcular la suma de excitatorias
    suma = sum(E)

    # 3. Comparar con el umbral
    if suma >= u:
        return 1
    else:
        return 0

# 1) OR de 2 entradas
def neurona_or2(a, b):
    return neurona_mcculloch_pitts([a, b], [], 1)

# 2) AND de 2 entradas (umbral = 2)
def neurona_and2(a, b):
    return neurona_mcculloch_pitts([a, b], [], 2)

# 3) DELAY / BUFFER (simple neurona que copia la entrada)
def neurona_delay(a):
    return neurona_mcculloch_pitts([a], [], 1)

# 4) NOT
def neurona_not(a):
    return neurona_mcculloch_pitts([], [a], 0)

# 5) MAJORITY de 3
def neurona_majority3(a, b, c):
    return neurona_mcculloch_pitts([a, b, c], [], 2)

# 6) e1 AND (NOT i1) -> excitatoria e1, inhibitoria i1, umbral 1
def neurona_e_and_not_i1(e1, i1):
    return neurona_mcculloch_pitts([e1], [i1], 1)

# 7) NOR de 2 entradas (NOT OR) -> dispara solo si ambas entradas 0
def neurona_nor2(a, b):
    return neurona_mcculloch_pitts([], [a, b], 0)

# 8) AND de 3 entradas (umbral = 3)
def neurona_and3(a, b, c):
    return neurona_mcculloch_pitts([a, b, c], [], 3)

# 9) OR de 3 entradas (umbral = 1)
def neurona_or3(a, b, c):
    return neurona_mcculloch_pitts([a, b, c], [], 1)

# 10) NOT e1 or e1
def neurona_not1_or_e1(e):
    return neurona_mcculloch_pitts([e], [], 0)

# 11) MAJORITY de 5 entradas
def neurona_majority5(a, b, c, d, e):
    return neurona_mcculloch_pitts([a, b, c, d, e], [], 3)

# 12) AND(a,b,NOT(i1),NOT(i2))
def neurona_mix_exc2_inh2(a, b, i1, i2):
    return neurona_mcculloch_pitts([a, b], [i1, i2], 2)

# -----
# Función auxiliar para probar todas las combinaciones de entradas y mostrar tablas
# -----
def probar_tabla(func, n_inputs, nombres=None):
    """
    Ejecuta todas las combinaciones binarias para una función 'func' que recibe n_inputs args.
    Imprime la tabla de verdad.
    """
    if nombres is None:
        nombres = [f"x{i+1}" for i in range(n_inputs)]
    # header
    header = " ".join(nombres) + " | out"
    print(header)
    print("-" * len(header))
    # iterate
    for val in range(2 ** n_inputs):
        bits = [(val >> (n_inputs - 1 - i)) & 1 for i in range(n_inputs)]
        out = func(*bits)
        print(f"{", ".join(str(b) for b in bits)} | " + str(out))
    print()

# -----
# Ejecutar comprobaciones (tablas) para cada neurona
# -----
if __name__ == "__main__":
    print("1) OR (2 inputs)")
    probar_tabla(neurona_or2, 2, ["a", "b"])

    print("2) AND (2 inputs)")
    probar_tabla(neurona_and2, 2, ["a", "b"])

    print("3) DELAY / BUFFER (1 input)")
    probar_tabla(neurona_delay, 1, ["a"])

    print("4) NOT (1 input)")
    probar_tabla(neurona_not, 1, ["a"])

    print("5) MAJORITY (3 inputs) -> dispara si >= 2 activas")
    probar_tabla(neurona_majority3, 3, ["a", "b", "c"])

    print("6) e1 AND (NOT i1) (2 inputs: e1, i1)")
    probar_tabla(neurona_e_and_not_i1, 2, ["e1", "i1"])

    print("7) NOR (2 inputs) -> NOT(OR)")
    probar_tabla(neurona_nor2, 2, ["a", "b"])

    print("8) AND (3 inputs)")
    probar_tabla(neurona_and3, 3, ["a", "b", "c"])

    print("9) OR (3 inputs)")
    probar_tabla(neurona_or3, 3, ["a", "b", "c"])

    print("10) NEURONA 10")
    probar_tabla(neurona_not1_or_e1, 1, ["e"])

    print("11) NEURONA 11")
    probar_tabla(neurona_majority5, 5, ["a", "b", "c", "d", "e"])

    print("12) NEURONA 12")
    probar_tabla(neurona_mix_exc2_inh2, 4, ["a", "b", "i1", "i2"])
```

EXPLICACIÓN:

Primero, se define la función

neurona_mcculloch_pitts, que representa una neurona artificial básica. Esta neurona recibe entradas excitatorias, entradas inhibitorias y un umbral. Su comportamiento es:

- Si alguna entrada inhibitoria está activa, la neurona no se activa.
- Si no hay inhibición, se suman las entradas excitatorias.
- Si esa suma es mayor o igual al umbral, la neurona se activa (1); en caso contrario, se mantiene apagada (0).

Después se definen las funciones de cada una de las 12 neuronas:

- La **neurona_or2** tiene dos entradas excitatorias y un umbral de 1. Esto significa que dispara si al menos una de las dos entradas está activa. En otras palabras, implementa una compuerta OR de 2.
- La **neurona_and2** recibe dos entradas excitatorias y su umbral es 2. Solo dispara cuando ambas entradas son 1, funcionando como una compuerta AND de 2.
- La **neurona_delay** se construye con una única entrada excitatoria y un umbral de 1. Su función es simplemente copiar la entrada en la salida, por lo que actúa como un buffer o retardo.
- La **neurona_not** utiliza una entrada inhibitoria. Si esta entrada vale 1, la salida se bloquea en 0, mientras que si la entrada es 0, la salida pasa a ser 1. Con este comportamiento, implementa una compuerta NOT.
- La **neurona_majority3** recibe tres entradas excitatorias y se activa con un umbral de 2. Esto implica que dispara si al menos dos de las tres entradas están activas, representando una función de mayoría.
- La **neurona_and_not** combina dos entradas excitatorias y una inhibitoria con un umbral de 1. Dispara si alguna de las excitatorias está activa y al mismo tiempo la inhibitoria no lo está, lo que corresponde a un AND con inhibición.

```

1) OR (2 inputs)
a b | out
-----
0 0 | 0
0 1 | 1
1 0 | 1
1 1 | 1

2) AND (2 inputs)
a b | out
-----
0 0 | 0
0 1 | 0
1 0 | 0
1 1 | 1

3) DELAY / BUFFER (1 input)
a | out
-----
0 | 0
1 | 1

4) NOT (1 input)
a | out
-----
0 | 1
1 | 0

5) MAJORITY (3 inputs) -> dispara si
a b c | out
-----
0 0 0 | 0
0 0 1 | 0
0 1 0 | 0
0 1 1 | 1
1 0 0 | 0
1 0 1 | 1
1 1 0 | 1
1 1 1 | 1

6) e1 AND (NOT i1) (2 inputs: e1,i1)
e1 i1 | out
-----
0 0 | 0
0 1 | 0
1 0 | 1
1 1 | 0

7) NOR (2 inputs) -> NOT(OR)
a b | out
-----
0 0 | 1
0 1 | 0
1 0 | 0
1 1 | 0

8) AND (3 inputs)
a b c | out
-----
0 0 0 | 0
0 0 1 | 0
0 1 0 | 0
0 1 1 | 0
1 0 0 | 0
1 0 1 | 0
1 1 0 | 0
1 1 1 | 1

9) OR (3 inputs)
a b c | out
-----
0 0 0 | 0
0 0 1 | 1
0 1 0 | 1
0 1 1 | 1
1 0 0 | 1
1 0 1 | 1
1 1 0 | 1
1 1 1 | 1

10) NEURONA 10
e | out
-----
0 | 1
1 | 1

11) NEURONA 11
a b c d e | out
-----
0 0 0 0 0 | 0
0 0 0 0 1 | 0
0 0 0 1 0 | 0
0 0 0 1 1 | 0
0 0 1 0 0 | 0
0 0 1 0 1 | 0
0 0 1 1 0 | 1
0 0 1 1 1 | 1
0 1 0 0 0 | 0
0 1 0 0 1 | 0
0 1 0 1 0 | 0
0 1 0 1 1 | 1
0 1 1 0 0 | 0
0 1 1 0 1 | 1
0 1 1 1 0 | 1
0 1 1 1 1 | 1
1 0 0 0 0 | 0
1 0 0 0 1 | 0
1 0 0 1 0 | 0
1 0 0 1 1 | 1
1 0 1 0 0 | 0
1 0 1 0 1 | 1
1 0 1 1 0 | 1
1 0 1 1 1 | 1
1 1 0 0 0 | 0
1 1 0 0 1 | 1
1 1 0 1 0 | 1
1 1 0 1 1 | 1
1 1 1 0 0 | 1
1 1 1 0 1 | 1
1 1 1 1 0 | 1
1 1 1 1 1 | 1

12) NEURONA 12
a b i1 i2 | out
-----
0 0 0 0 | 0
0 0 0 1 | 0
0 0 1 0 | 0
0 0 1 1 | 0
0 1 0 0 | 0
0 1 0 1 | 0
0 1 1 0 | 0
0 1 1 1 | 0
1 0 0 0 | 0
1 0 0 1 | 0
1 0 1 0 | 0
1 0 1 1 | 0
1 1 0 0 | 1
1 1 0 1 | 0
1 1 1 0 | 0
1 1 1 1 | 0

```

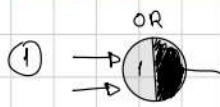
- La **neurona_nor2** está formada por dos entradas inhibitorias y un umbral de 1. Solo dispara cuando ninguna de las entradas es 1, es decir, cuando ambas están en 0. Implementa una compuerta NOR de 2.
- La **neurona_and3** utiliza tres entradas excitatorias y un umbral de 3. De esta forma, dispara únicamente cuando las tres entradas son iguales a 1, funcionando como una compuerta AND de 3.
- La **neurona_or3** también recibe tres entradas excitatorias, pero en este caso el umbral es 1. Esto significa que dispara si cualquiera de las tres entradas está activa, cumpliendo la función de un OR de 3.
- La **neurona_note1_or_e1** recibe una sola entrada excitatoria con umbral 0. Esto provoca que siempre dispare sin importar el valor de la entrada, representando así una neurona constante o de sesgo (*bias*).
- La **neurona_majority5** se construye con cinco entradas excitatorias y un umbral de 3. Dispara cuando al menos tres de las cinco entradas están activas, implementando una función de mayoría de 5.
- Por último, la **neurona_mix_exc2_inh2** recibe dos entradas excitatorias y dos inhibitorias con un umbral de 2. Se activa únicamente cuando ambas excitatorias están encendidas y ninguna inhibitoria lo bloquea, representando una función AND con inhibición.

Con este modelo se define la función **probar_tabla** la cual genera toda la tabla de verdad de una neurona, para después probar todas las combinaciones posibles y muestra la salida.

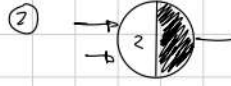
En la parte principal del programa se ejecuta **probar_tabla** para todas las neuronas 1–12, generando sus tablas de verdad automáticamente.

Diseño de las 12 neuronas

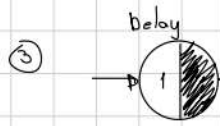
Tablas de verdad neuronas



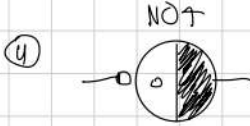
e_1	e_2	sal
0	0	0
0	1	1
1	0	1
1	1	1



e_1	e_2	sal
0	0	0
0	1	0
1	0	0
1	1	1



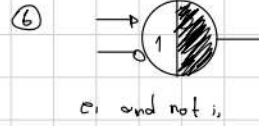
e_1	sal
0	0
1	1



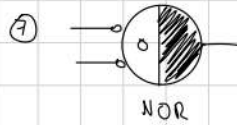
i_1	sal
0	1
1	0



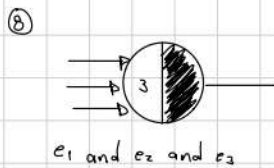
e_1	e_2	e_3	sal
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



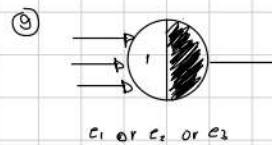
e_1	i_1	sal
0	0	0
0	1	0
1	0	1
1	1	0



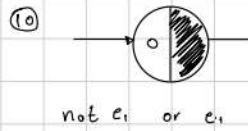
i_1	i_2	sal
0	0	1
0	1	0
1	0	0
1	1	0



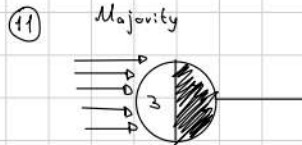
e_1	e_2	e_3	sal
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



e_1	e_2	e_3	sal
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



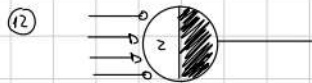
e_1	sal
0	1
1	1



(11)

	e_1	e_2	e_3	e_4	e_5	sal
0	0	0	0	0	0	0
1	0	0	0	0	1	0
2	0	0	0	1	0	0
3	0	0	0	1	1	0
4	0	0	1	0	0	0
5	0	0	1	0	1	0
6	0	0	1	1	0	0
7	0	0	1	1	1	1
8	0	1	0	0	0	0
9	0	1	0	0	1	0
10	0	1	0	1	0	0
11	0	1	0	1	1	1
12	0	1	1	0	0	0
13	0	1	1	0	1	1
14	0	1	1	1	0	1
15	0	1	1	1	1	1

	e_1	e_2	e_3	e_4	e_5	sal
16	1	0	0	0	0	0
17	1	0	0	0	1	0
18	1	0	0	1	0	0
19	1	0	0	1	1	1
20	1	0	1	0	0	0
21	1	0	1	0	1	1
22	1	0	1	1	0	1
23	1	0	1	1	1	1
24	1	1	0	0	0	0
25	1	1	0	0	1	1
26	1	1	0	1	0	1
27	1	1	0	1	1	1
28	1	1	1	0	0	1
29	1	1	1	0	1	1
30	1	1	1	1	0	1
31	1	1	1	1	1	1



i	e_1	e_2	i_2	sal
0	0	0	0	0
1	0	0	1	0
2	0	0	1	0
3	0	0	1	0
4	0	1	0	0
5	0	1	0	0
6	0	1	1	1
7	0	1	1	0
8	1	0	0	0
9	1	0	0	0
10	1	0	1	0
11	1	0	1	0
12	1	1	0	0
13	1	1	0	0
14	1	1	1	0
15	1	1	1	0

Neurona Compuerta XOR

```

1 def neurona_mcculloch_pitts(E, I, u):
2     """
3     Simula una neurona de McCulloch y Pitts.
4
5     Parámetros:
6     E : lista de enteros (0 o 1) -> entradas excitatorias
7     I : lista de enteros (0 o 1) -> entradas inhibitorias
8     u : entero (umbral)
9
10    Retorna:
11    0 -> la neurona permanece apagada
12    1 -> la neurona se activa
13    """
14    # 1. Revisar entradas inhibitorias: si alguna está activa, la neurona no dispara
15    if 1 in I:
16        return 0
17
18    # 2. Calcular la suma de excitatorias
19    suma = sum(E)
20
21    # 3. Comparar con el umbral
22    if suma >= u:
23        return 1
24    else:
25        return 0
26
27
28 def xor_myp(A, B):
29     """
30     Implementa XOR usando neuronas McCulloch-Pitts (MyP) con
31     entradas excitatorias e inhibitorias.
32     Estructura:
33     H1: excitatoria A, inhibitoria B, umbral 1 -> dispara solo si A=1 y B=0
34     H2: excitatoria B, inhibitoria A, umbral 1 -> dispara solo si A=0 y B=1
35     Out: excitatorias H1,H2, sin inhibitorias, umbral 1 (OR)
36     Devuelve (h1, h2, out)
37     """
38    # H1 = A AND (NOT B)
39    n1 = neurona_mcculloch_pitts(E=[A], I=[B], u=1)
40
41    # H2 = (NOT A) AND B
42    n2 = neurona_mcculloch_pitts(E=[B], I=[A], u=1)
43
44    # Out = H1 OR H2
45    n3 = neurona_mcculloch_pitts(E=[n1, n2], I=[], u=1)
46
47    return n1, n2, n3
48
49
50 if __name__ == "__main__":
51     print("A B | N1 N2 | XOR")
52     print("-----")
53     for A in (0, 1):
54         for B in (0, 1):
55             n1, n2, n3 = xor_myp(A, B)
56             print(f"{A} {B} | {n1} {n2} | {n3}")
57

```

EXPLICACIÓN:

La función `neurona_mcculloch_pitts` recibe tres parámetros: las entradas excitatorias (E), las entradas inhibitorias (I) y un umbral (u). Su funcionamiento es simple:

1. Si alguna entrada inhibitoria está activa, la neurona se apaga automáticamente.
2. Si no hay inhibiciones, se suman las entradas excitatorias.
3. Si esa suma es mayor o igual al umbral, la neurona se activa (1); en caso contrario, permanece apagada (0).

Con este modelo se define la función `xor_myp`, que combina tres neuronas para obtener el comportamiento de XOR:

- La primera (H1) se activa solo cuando $A=1$ y $B=0$.
- La segunda (H2) se activa solo cuando $A=0$ y $B=1$.
- La tercera (Out) recibe como entradas a H1 y H2 y funciona como un OR, produciendo así la salida final de XOR.

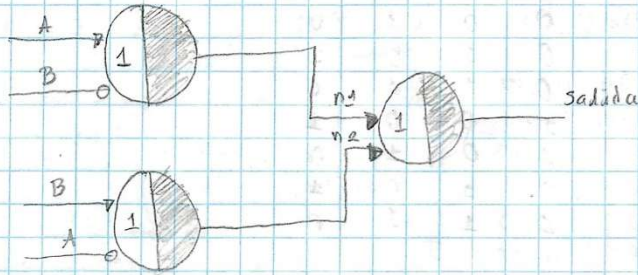
En la parte principal del programa se prueban todas las combinaciones de valores de A y B, mostrando en pantalla los resultados intermedios y el valor final de la operación XOR.

A	B	N1	N2	XOR
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

Diseño de la compuerta XOR

Diseñen y programen la operación XOR con neuronas de M.P. interconectadas. Además, imprimen las salidas para cada posible combinación de entradas.

Diseño de la operación XOR



Primera neurona

Segunda neurona

XOR (Tercera neurona)

A	B	n1
0	0	0
0	1	0
1	0	1
1	1	0

A	B	n2
0	0	0
0	1	1
1	0	0
1	1	0

n1	n2	S
0	0	0
0	1	1
1	0	1
0	0	0

Neurona medio sumador Half Adder

```

1 def neurona_mcculloch_pitts(E, I, u):
2     """
3     Simula una neurona de McCulloch y Pitts.
4
5     Parámetros:
6     E : lista de enteros (0 o 1) -> entradas excitatorias
7     I : lista de enteros (0 o 1) -> entradas inhibitorias
8     u : entero (umbral)
9
10    Retorna:
11    0 -> la neurona permanece apagada
12    1 -> la neurona se activa
13    """
14    # 1. Revisar entradas inhibitorias: si alguna está activa, la neurona no dispara
15    if 1 in I:
16        return 0
17
18    # 2. Calcular la suma de excitatorias
19    suma = sum(E)
20
21    # 3. Comparar con el umbral
22    if suma >= u:
23        return 1
24    else:
25        return 0
26
27
28 def xor_myp(A, B):
29     """
30     Implementa XOR usando neuronas McCulloch-Pitts (MyP) con
31     entradas excitatorias e inhibitorias.
32     Estructura:
33     N1: excitatoria A, inhibitoria B, umbral 1 -> dispara solo si A=1 y B=0
34     N2: excitatoria B, inhibitoria A, umbral 1 -> dispara solo si A=0 y B=1
35     N3: excitatorias N1,N2, sin inhibitorias, umbral 1 (OR)
36     Devuelve (n1, n2, n3) donde n3 es A XOR B.
37     """
38    # N1 = A AND (NOT B)
39    n1 = neurona_mcculloch_pitts(E=[A], I=[B], u=1)
40
41    # N2 = (NOT A) AND B
42    n2 = neurona_mcculloch_pitts(E=[B], I=[A], u=1)
43
44    # N3 = N1 OR N2
45    n3 = neurona_mcculloch_pitts(E=[n1, n2], I=[], u=1)
46
47    return n1, n2, n3
48
49
50 def half_adder(A, B):
51     """
52     Medio sumador usando neuronas MyP.
53     Retorna (S, C, n1, n2) donde:
54     S = suma (A XOR B)
55     C = carry (A AND B)
56     n1,n2 son señales intermedias del XOR
57     """
58    n1, n2, S = xor_myp(A, B)
59    C = neurona_mcculloch_pitts(E=[A, B], I=[], u=2) # AND
60    return S, C, n1, n2
61
62
63 if __name__ == "__main__":
64     print("A B | N1 N2 | S (XOR) C (AND)")
65     print("-----")
66     for A in (0, 1):
67         for B in (0, 1):
68             S, C, n1, n2 = half_adder(A, B)
69             print(f"{A} {B} | {n1} {n2} | {S} {C}")
70

```

EXPLICACIÓN:

Primero, se define la función

neurona_mcculloch_pitts, que representa una neurona artificial básica. Esta neurona recibe entradas excitatorias, entradas inhibitorias y un umbral. Su comportamiento es:

- Si alguna entrada inhibitoria está activa, la neurona no se activa.
- Si no hay inhibición, se suman las entradas excitatorias.
- Si esa suma es mayor o igual al umbral, la neurona se activa (1); en caso contrario, se mantiene apagada (0).

Después, la función *xor_myp* implementa la operación lógica **XOR** mediante tres neuronas:

- N1 se activa cuando A=1 y B=0.
- N2 se activa cuando A=0 y B=1.
- N3 recibe como entradas a N1 y N2, y funciona como un OR, dando como salida la operación XOR.

Con esta base se construye la función *half_adder*, que corresponde a un **medio sumador**. Este circuito recibe dos bits de entrada (A y B) y produce dos salidas:

- S es la suma, obtenida con XOR.
- C es el acarreo, calculado con una neurona que implementa AND (umbral 2 con entradas A y B).

Finalmente, en la parte principal del programa se muestran todas las combinaciones posibles de entradas (0 y 1), junto con los resultados de las señales intermedias y las salidas del medio sumador (S y C).

A	B	N1	N2	S (XOR)	C (AND)
0	0	0	0	0	0
0	1	0	1	1	0
1	0	1	0	1	0
1	1	0	0	0	1

Diseño del Half Adder

Ahora implementen un medio sumador **Half Adder** el cual combina dos entradas A, B y produce:

- Suma (S) $\rightarrow$ es un XOR ($A \oplus B$)
- Carry (C) $\rightarrow$ es un AND ($A \wedge B$)

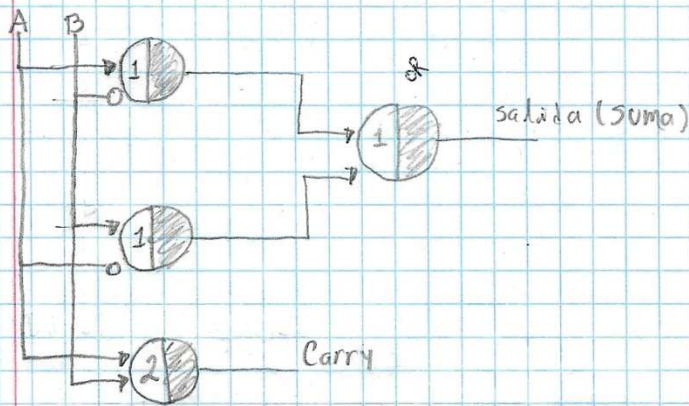
Esto obliga a usar al menos dos redes de neuronas M y P interconectadas (una para XOR y otra para AND).

La tabla de verdad es:

A	B	Suma ($S = A \oplus B$)	Carry ($C = A \wedge B$)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

La salida **S** indica la suma binaria de un solo bit.
La salida **C** indica si hay acarreo (cuando ambos son 1).

Diseño del Half Adder:



Conclusiones

Aguirre Lanto Víctor Manuel

En mi caso, lo que más me llamó la atención fue entender cómo una neurona puede simular operaciones lógicas básicas como AND, OR. Al inicio pensé que era algo muy complicado, pero al ver que todo se reduce a pesos y un sesgo, me di cuenta que en realidad es algo sencillo.

Gasca Fragoso Pedro

Lo que más se me dificultó fue comprender por qué el sesgo debía ser negativo en el AND, o por qué no podía ser igual a cero en ciertos casos. Me costó entender la idea del umbral y de que todo depende de la función escalón.

Guevara Badillo Areli Alejandra

Esta práctica me ayudó a entender cómo llegar a representar con una neurona tan simple como la de McCulloch y Pitts, operaciones más complejas como son las operaciones lógicas y el medio sumador. Lo más retador de esta práctica fue comprender cómo funcionaban las entradas excitatorias, inhibitorias y el umbral, y lograr las combinaciones adecuadas para conseguir las salidas esperadas.

Montiel Toro Arael de Jesús

Yo tuve más problemas al momento de entender el XOR y cómo se relaciona el álgebra booleana, después me gustó el tema de perceptrones multicapa, ayuda mucho con el aprendizaje supervisado.

Ramírez Lozano Gael Martin

Lo que más me llamó la atención fue la parte del Half adder. Me pareció sorprendente que se pudieran implementar una suma binaria simplemente con neuronas artificiales, me gusta mucho la relación que tienen con circuitos digitales.